# Fundamentals / Foundation of Data Science

Data Science 9CFU

Computer Science 6CFU

Indro Spinelli

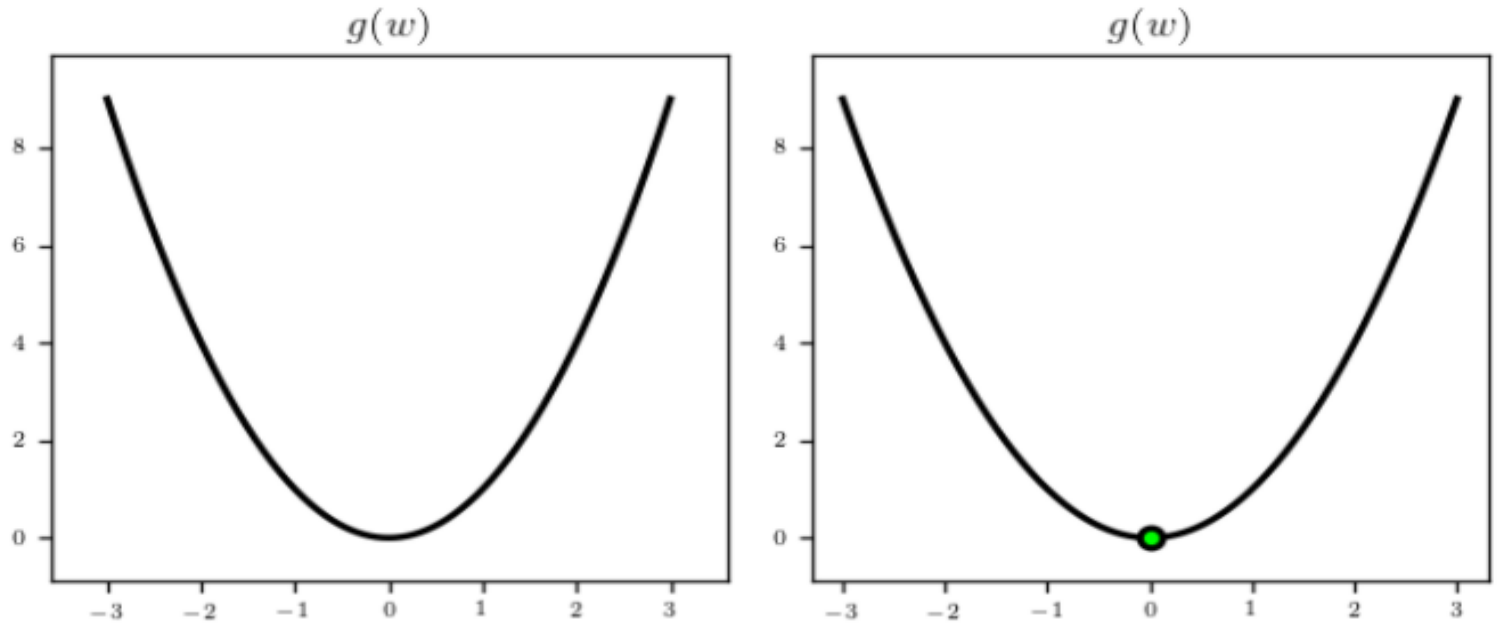
Sapienza University of Rome

# Optimization

Optimization refers to the task of either minimizing or maximizing some function $g(\mathbf{w})$ by altering $\mathbf{w}$.

This function is called the objective function. When we are interested in the minimization it can be called also loss function.

The value that minimizes or maximizes a function is denoted with *.


$g(w)$
8
6
4
2
0
−3
−2
−1
0
1
2
3
$g(w)$
8
6
4
2
0
−3
−2
−1
0
1
2
3

# Quadratic function

The smallest point of a quadratic function $g(w) = w^2$ is called the *zero-order definition of a global minimum point*.

$$g(w^\star) \leq g(w) \text{ for all } w.$$

If we flip the parabola upside down, the point $w^\star = 0$ becomes its global maximum

$$g(w^\star) \geq g(w) \text{ for all } w.$$

These concepts of *minima* and *maxima* of a function are always related to each via multiplication by ***-1***.

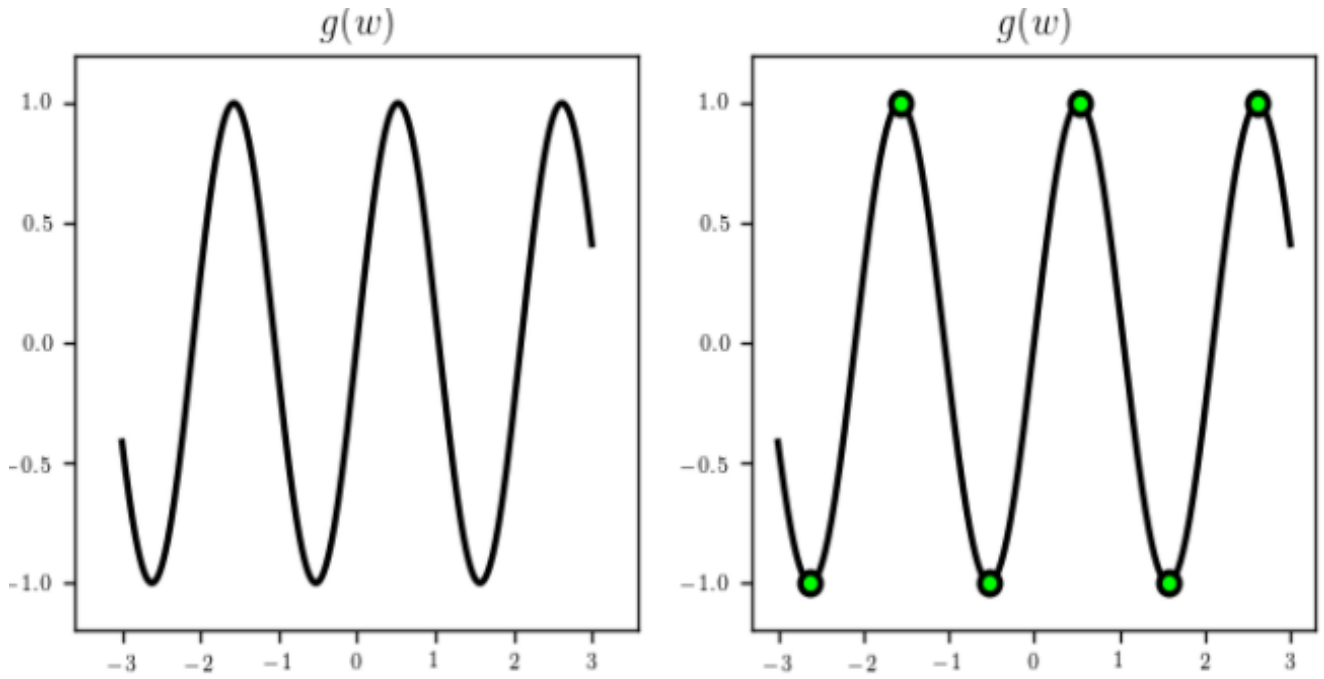
# Quadratic function

To express our pursuit of a global *maxima* of a function we write:

$$\underset{\mathbf{w}}{\text{maximize}} \quad g(\mathbf{w}).$$

Since any point that is a minima of a function $\boldsymbol{g}$ is a maxima of the function $\boldsymbol{-g}$, and vice-versa.

$$\underset{\mathbf{w}}{\text{maximize}} \quad g(\mathbf{w}) = -\underset{\mathbf{w}}{\text{minimize}} \quad g(\mathbf{w}).$$


$g(w)$
1.0
0.5
0.0
−0.5
−1.0
−3
−2
−1
0
1
2
3
$g(w)$
1.0
0.5
0.0
−0.5
−1.0
−3
−2
−1
0
1
2
3

# Sinusoid function

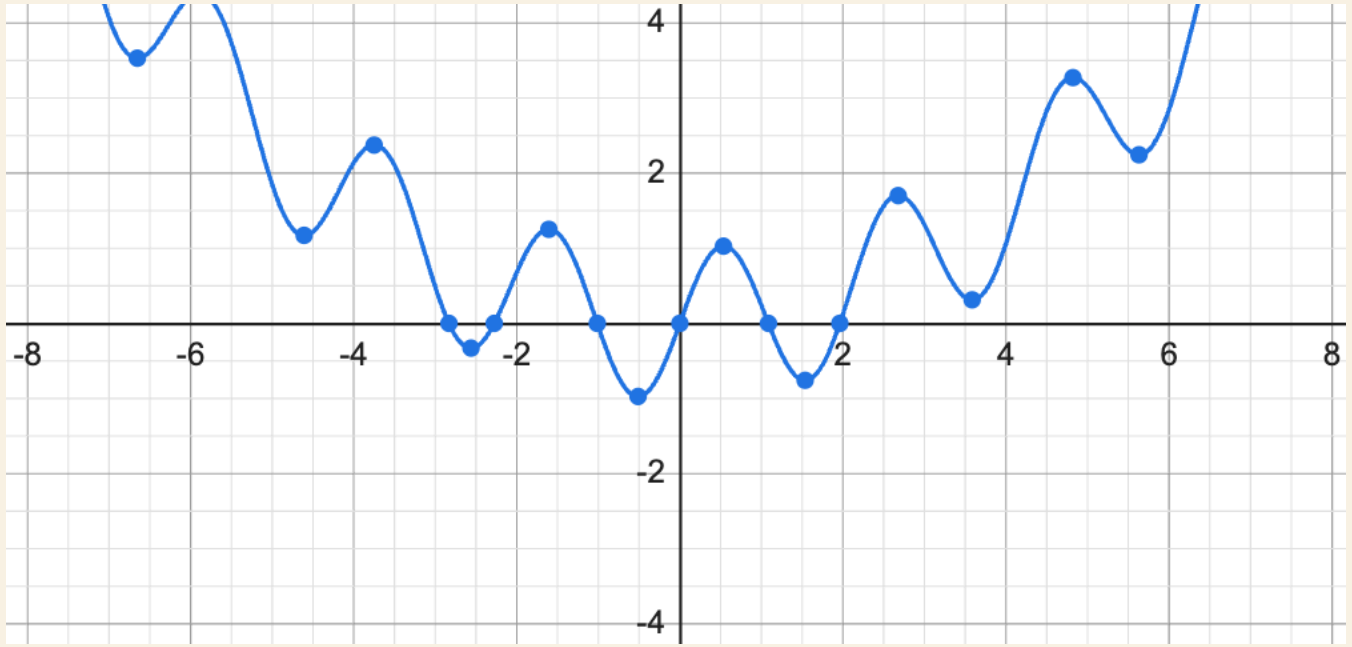
If we have a sinuoid $g(w) = \sin(2w)$

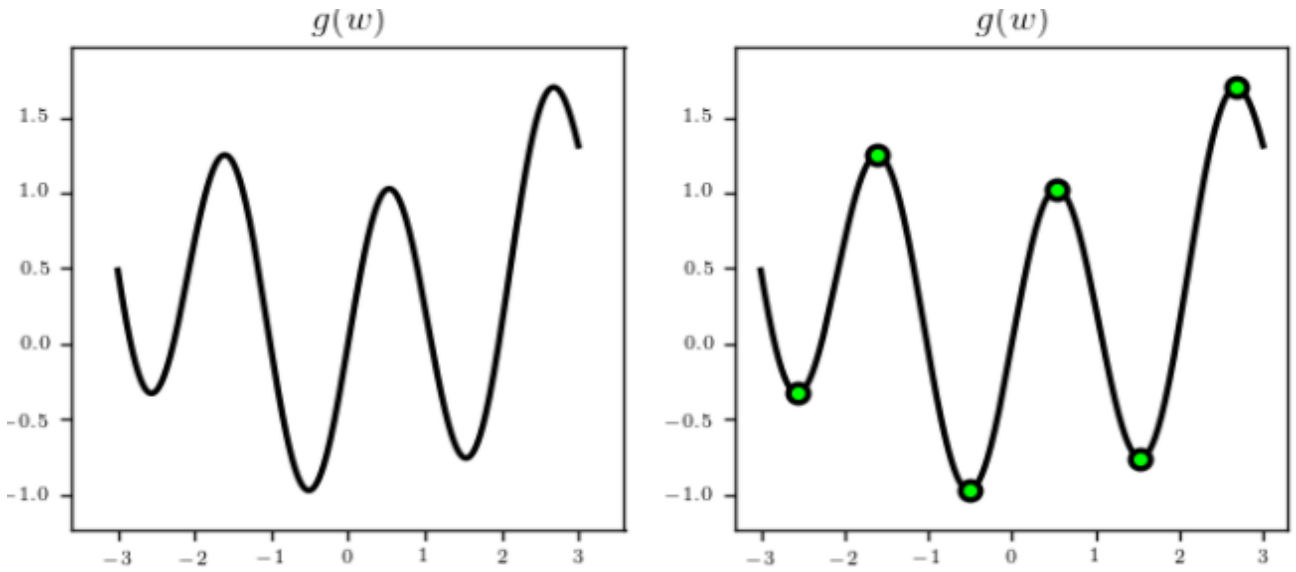
We can clearly see that - over the range we have plotted the function - that there are two global minima and two global maxima (marked by green dots in the right panel).

In reality, since the function is periodic it has an infinite number of global minima and maxima.

# What if we put them together?

$$g(w) = \sin(3w) + 0.1w^2$$




$g(w)$
1.5
1.0
0.5
0.0
−0.5
−1.0
−3
−2
−1
0
1
2
3
$g(w)$
1.5
1.0
0.5
0.0
−0.5
−1.0
−3
−2
−1
0
1
2
3

# Local and Global

We have a global minimum at w = -0.5.

We have also minima (w= 1.5) and maxima (w=0.8, w=2.7) points that are only locally optimal.

A **local minimum** is a point in the domain of a function where the function value is smaller than (or equal to) the values at all nearby points, but not necessarily the smallest value overall.

$$g(w^\star) \leq g(w) \quad \text{for all } w \text{ near } w^\star$$

# Zero-Order

These are called zero-order conditions becouse thei involve the fuction itself.

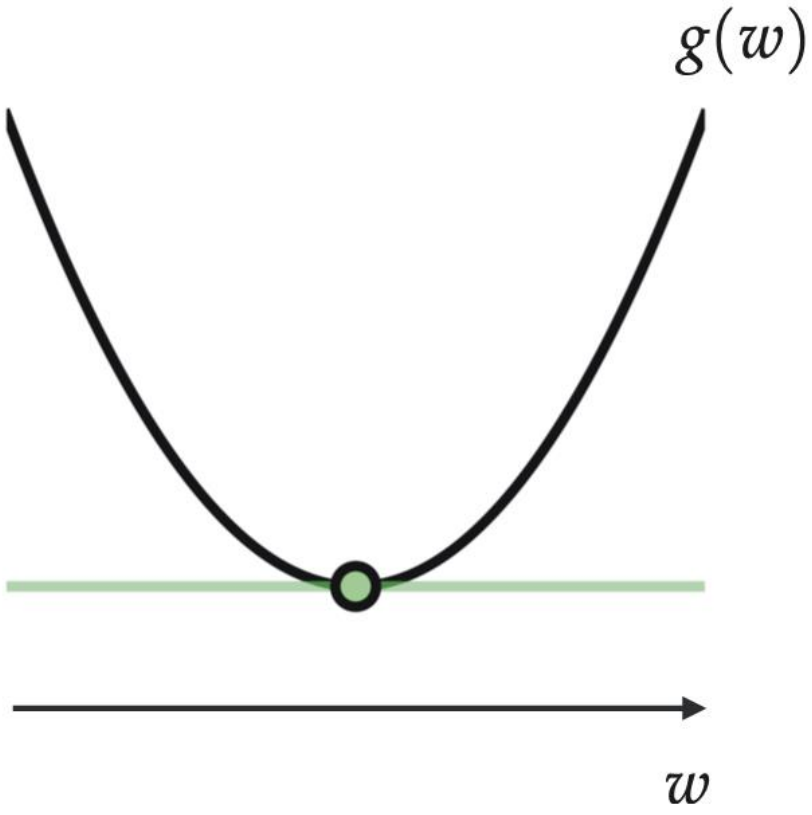
*First-order* definitions that involve the first derivative of a function and *second-order* definitions involving a function's second derivative.

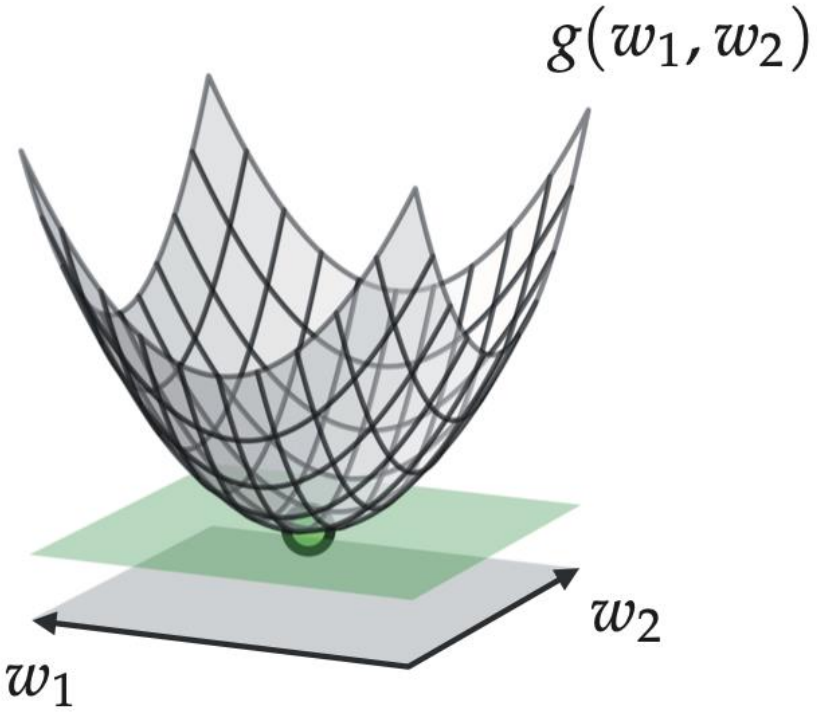
# First Order

We plot quadratic functions in both two and three dimensions, marking the global minimum in green.

Each panel also shows the first-order Taylor approximation (a tangent line or hyperplane) obtained from the function's derivative(s) at the minimum.

At the minimum, the tangent is perfectly flat!


$g(w)$
$w$


$g(w_1, w_2)$
$w_2$
$w_1$

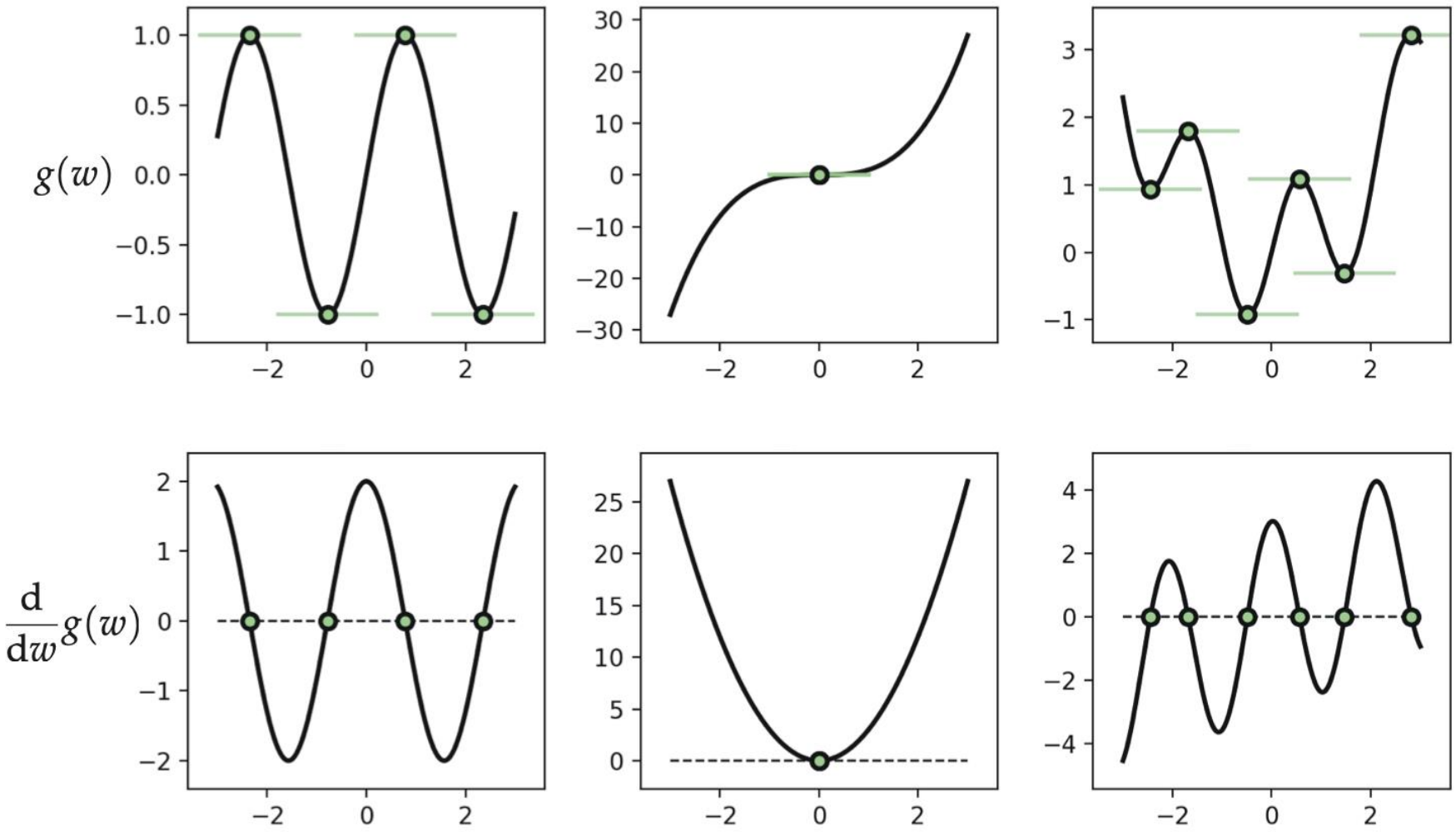
# First Order

This first-order behavior is universal: it holds for any function and in any input dimension.

At a minimum, the first derivatives are always zero.

Intuitively, minima lie at the valley floor, where the tangent line or hyperplane is perfectly flat, giving zero slope.

Any point where $\frac{\mathrm{d}}{\mathrm{d}w} g\left(v\right) = 0$ is a potential minimum.


$g(w)$
$\frac{\mathrm{d}}{\mathrm{d}w}g(w)$

# Multidimensionality

By analogy, in **N dimensions**, any point where **all partial derivatives of $g$ vanish** is a **candidate minimum**.

$$\frac{\partial}{\partial w_1} g(\mathbf{v}) = 0$$
$$\frac{\partial}{\partial w_2} g(\mathbf{v}) = 0$$
$$\vdots$$
$$\frac{\partial}{\partial w_N} g(\mathbf{v}) = 0$$

A system of $N$ equations like this is naturally called a **first-order system of equations**.

$$\nabla g\left(\mathbf{v}\right) = \mathbf{0}_{N\times 1}.$$

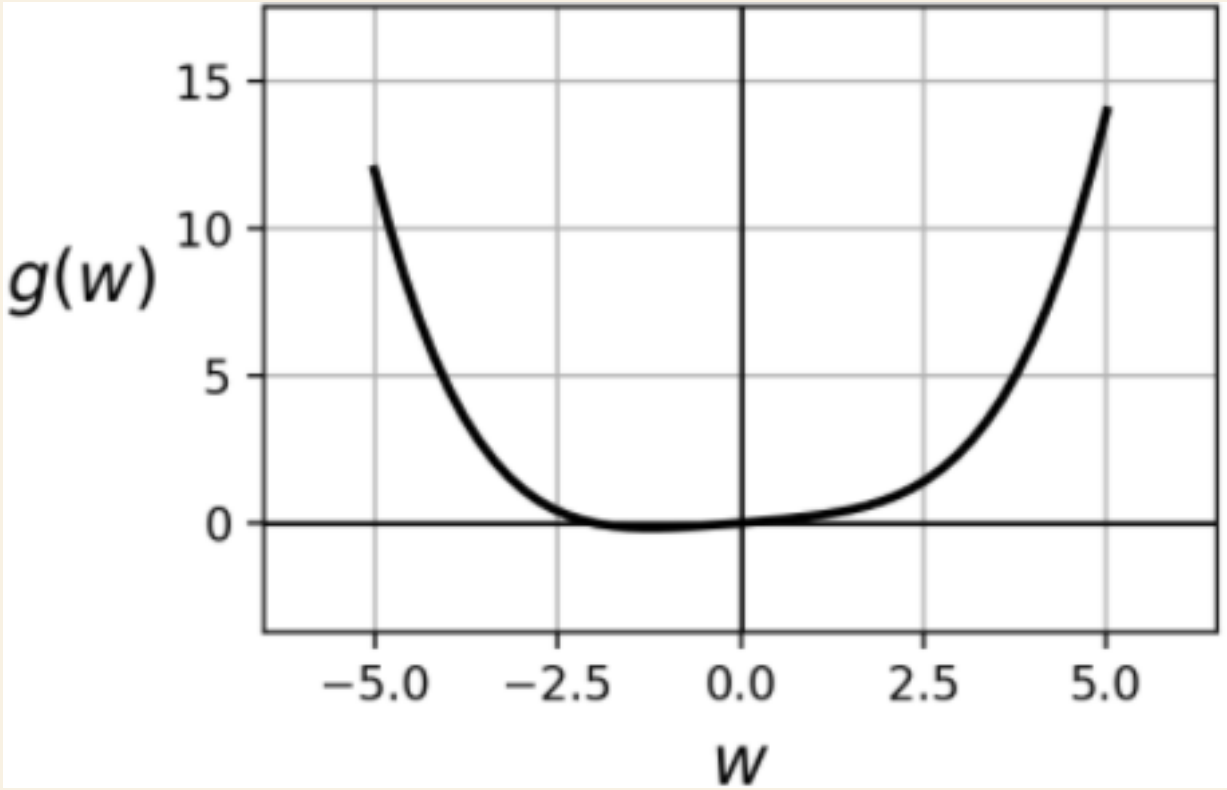
# Analytic Solution

The solution to this system of equations can be sometimes computed analytically (closed form).

For the central panel $g(w) = w^3$ the first order condition gives.

$g'(v) = 3v^2 = 0$ which we can identify as a saddle point for $v = 0$

# But it is not always so simple




$$w = \frac{\sqrt[3]{\sqrt{2031}-45}}{6^{\frac{2}{3}}} - \frac{1}{\sqrt[3]{6\left(\sqrt{2031}-45\right)}}$$

# Hyperplane

$$h(w_1, w_2, \ldots, w_N) = a + b_1 w_1 + b_2 w_2 + \cdots + b_N w_N$$

We can use compact vector notation to write:

$$h(w) = a + \mathbf{b}^\top \mathbf{w}$$

Notice that $h(w) = a + bw$ is the equation of a simple line.

It is a one-dimensional object living in a two-dimensional ambient space, where the input space (w) is itself one-dimensional. The same reasoning applies for the general case of N dimensions.

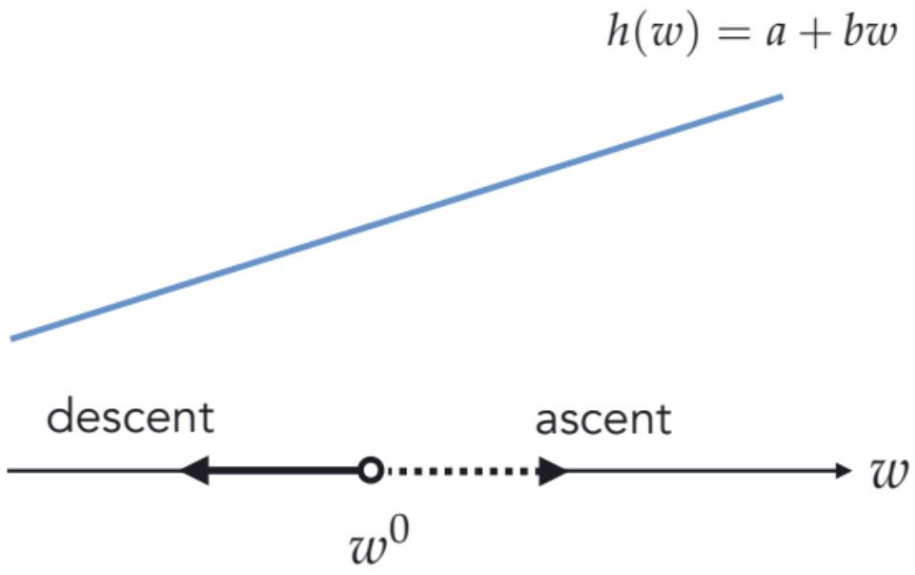
# From 1D to Multi-Dimensional Hyperplanes

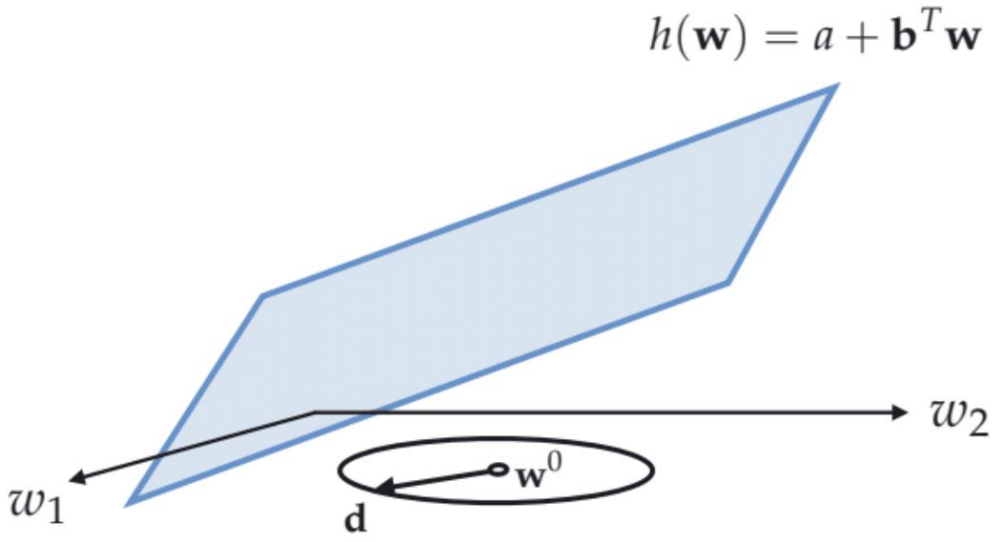
For a **one-dimensional hyperplane** ($N = 1$), the input space is also one-dimensional.

- At any point $w_0$ ,there are only **two directions** to move: left or right.
- Moving **right** ($+1$) increases $h$ → **ascent direction**.
- Moving **left** ($-1$) decreases $h$ → **descent direction**.

For **higher dimensions** ($N > 1$):

- There are **infinitely many directions** at each point.
- Some directions **increase** $h$(ascent), some **decrease** $h$ (descent), and some **leave** $h$ **unchanged**.


$h(w) = a + bw$
descent
ascent
$w$
$w^0$


$h(\mathbf{w}) = a + \mathbf{b}^T\mathbf{w}$
$w_2$
$w_1$
$\mathbf{w}^0$
$\mathbf{d}$

# Direction of Steepest Ascent

**Can we find the direction that produces the largest ascent (or descent)?**

- This is known as the **direction of steepest ascent** (or descent).

**Formalization**

At a given point $w_0$ ,we seek the **unit vector** $d$ that maximizes $h(w_0 + d)$:

$$\max_{d} \; h(w_0 + d) \quad \text{subject to} \quad \|d\| = 1$$

Intuition: $d$ points in the direction where $h$ increases most rapidly.

# Reducing the Steepest Ascent Problem

$$h(w_0 + d) = a + b^\top(w_0 + d) = a + b^\top w_0 + b^\top d$$

The first two terms are **constant with respect to** $d$.

Therefore, **maximizing** $h(w_0 + d)$ **is equivalent to maximizing** $b^\top d$.

$$b^\top d = \|b\| \, \|d\| \cos(\theta)$$

- $\| b \|$is constant with respect to $d$ & $\| d \| = 1$.

**The problem reduces to:** $\max_{\theta} \cos(\theta)$

# Steepest Ascent and Descent Directions

Of all unit directions $d = \frac{b}{\|b\|}$

provides the **steepest ascent**, where $\theta = 0$

Conversely, the unit direction $d = -\frac{b}{\|b\|}$

provides the **steepest descent**, where $\theta = \pi$ .

**Key insight:** The gradient vector $b$ points in the direction of maximum increase of $h$.

# The Gradient $\nabla g(w_0)$

The **gradient** of a function $g(w)$ at a point $w_0$ is the vector of all its partial derivatives:

$$\nabla g(w_0) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(w_0) \\ \frac{\partial g}{\partial w_2}(w_0) \\ \vdots \\ \frac{\partial g}{\partial w_n}(w_0) \end{bmatrix}$$

**Key Properties**

🚀 Direction of steepest ascent

📈 Magnitude = rate of increase

⊥ Orthogonal to level curves/surfaces

# The Gradient and Steepest Directions

**Local Linear Approximation**

A function: $g(w)$ can be approximated locally, around a point $w_0$ , with tangent hyperplane: $h(w) = g(w_0) + \nabla g(w_0)^\top (w - w_0)$

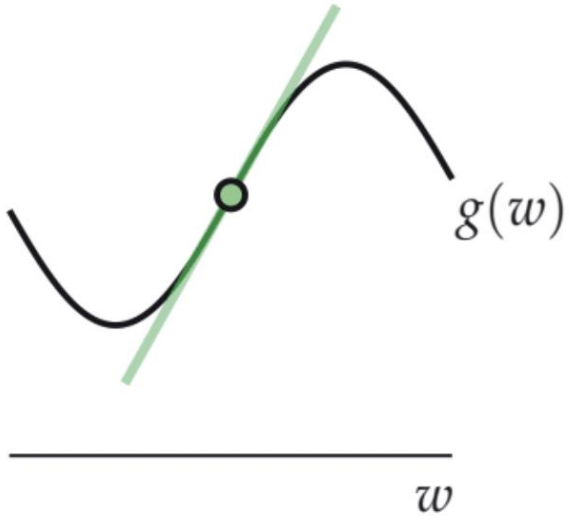
$h(w) = a + b^\top w$

$$a = g(w_0) - \nabla g(w_0)^\top w_0, \quad b = \nabla g(w_0)$$

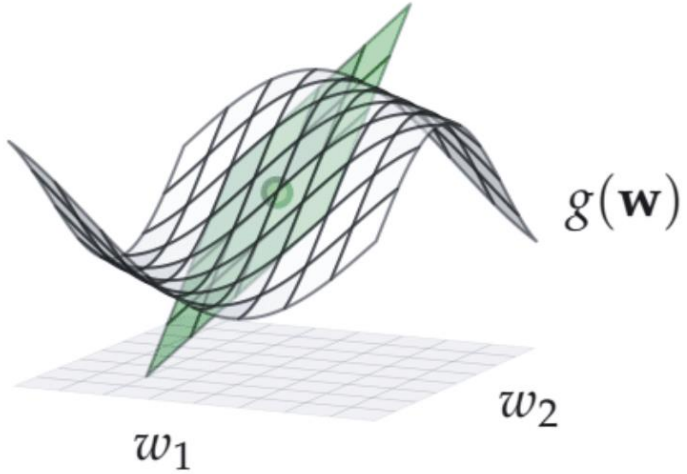
Linear form:

- b= $\nabla g(w_0)$ sets the *orientation* of the tangent plane (its slope).
- The term $-\nabla g(w_0)^T w_0 \ \ in\ a$ adjusts the *offset* so that the plane is not "floating" but exactly touches $g(w)$at $w_0$.

$$h(w) = g(w^0) + \frac{\mathrm{d}}{\mathrm{d}w} g(w^0) \left(w - w^0\right)$$




$$h(\mathbf{w}) = g(\mathbf{w}^0) + \nabla g(\mathbf{w}^0)^T \left(\mathbf{w} - \mathbf{w}^0\right)$$



# Steepest Ascent and Descent

The **direction of steepest ascent** at a point $w_0$ is given by the **gradient vector** $+\nabla g(w_0)$ ,which points in the direction where $g(w)$ increases most rapidly.

The **direction of steepest descent** is $-\nabla g(w_0)$ ,pointing where $g(w)$ decreases most rapidly.

# Why it works

Near $w_0$ ,the function $g(w)$ can be approximated by its **tangent hyperplane** $h(w)$.

The directions of maximum increase and decrease for $h(w)$ match those of $g(w)$.

This explains why following $+\nabla g(w)$or $-\nabla g(w)$ leads to ascent or descent.

# Practical Implications

Gradient information is widely used in optimization algorithms:

**Gradient Descent:** Move along $-\nabla g(w)$ to reach a local minimum.

**Gradient Ascent:** Move along $+\nabla g(w)$ to reach a local maximum.

Using the local linear approximation helps to understand the behavior of $g(w)$ near $w_0$ and guides step sizes and directions in iterative optimization methods.

# Computing Derivatives

Finding the analytical derivative of a complex function $g(w)$ can be difficult.

We can use a **numerical derivative calculator** at a user-defined point $w_0$ by choosing a small step $\epsilon$.

$$\left.\frac{dw}{dg}\right|_{w_0} \approx \frac{g(w_0+\epsilon)-g(w_0-\epsilon)}{2\epsilon}$$

# Important rules

**Chain Rule**

For a composite function $\mathrm{f}(g(w))$ the derivative with respect to w is:

$$\frac{d}{dw} f(g(w)) = f'(g(w)) \cdot g'(w)$$

**Product Rule**

For the product of two functions $\mathrm{f}(w) * g(w)$ the derivative with respect to w is:

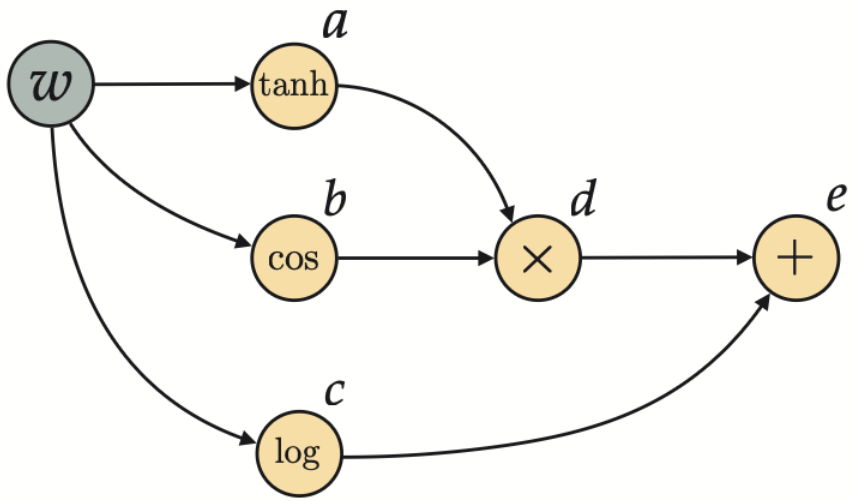
$$\frac{d}{dw}\big(f(w) \cdot g(w)\big) = f'(w)g(w) + f(w)g'(w)$$

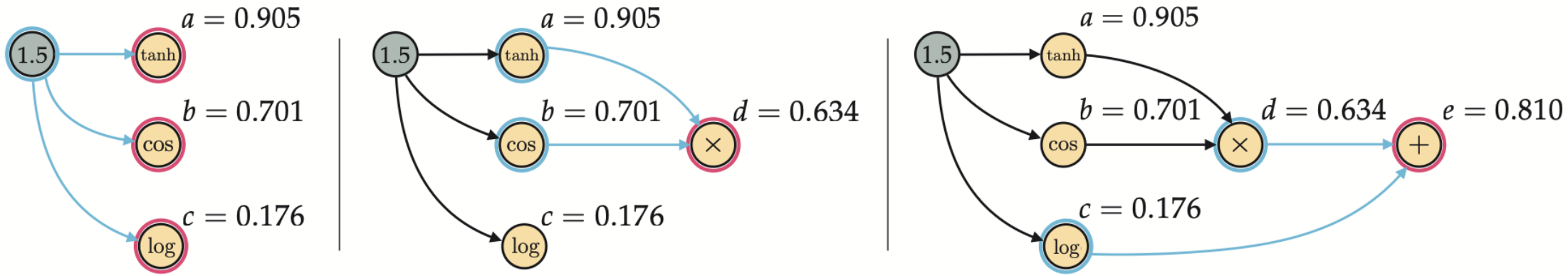
# Decomposing Functions with Computation Graphs

Virtually any function $g$, expressed via an algebraic formula, can be broken down into a combination of **elementary functions** (e.g., sin(·), e^(·), log(·), etc.) and **operations** (e.g., addition, multiplication, composition, etc.).

A very useful way to organize this decomposition is through a **computation graph** which enables programmatic evaluation of the function.

$$g(w) = \tanh(w)\cos(w) + \log(w).$$



# Reading the graph

Start **left to right** from the input node w.

End at the full computation of g(w) on the right.

**Yellow nodes:** represent individual functions or operations.

**Gray node:** input w.

**Edges/arrows:** show the flow of computation.

# Graph Topology

**Parent and child terms:**

**Parent:** the node where an edge originates.

**Child:** the node where the edge points.

Nodes can be **both parent and child** depending on context.

- **Example from the graph of g(w):**
  - Input node w (gray) is **parent** to nodes a, b, and c.
  - Nodes a and b are **parents** to node d.

# Child Nodes as Functions of Their Parents

$$g(w) = \tanh(w)\cos(w) + \log(w).$$

a = tanh(w)
b = cos(w)
c = log(w)
d = a * b
e = c + d

Each child node is a function of its parent(s).

- Example: a = a(w) since w is the only parent.
- Example: d = d(a, b) since a and b are parents.

Unravelling all nodes: every node is ultimately a function of the input w.

- Example: d(w) = tanh(w) * cos(w)

The computation flows forward through the graph in sets of parent–child nodes.

# Forward Mode of Automatic Differentiation

The same idea applies to **gradient computation:**

- Sweep forward through the graph.
- Form and evaluate the gradient of each node w.r.t. the original input.
- Original function values are naturally evaluated at each node during this process.

**Advantages of forward mode:**

- Easily programmable.
- Offloads tedious gradient computation to a computer.
- Provides **exact derivatives**, unlike numerical differentiation.

# In short

**Goal:** Evaluate the derivative d/dw g(w) using **forward mode AD**.

**Method**

- Traverse the computation graph (previously illustrated in Figure B.3) **left to right**, like function evaluation.
- Compute the derivative at each node recursively.

Forward mode AD allows simultaneous evaluation of the function and its derivative in a systematic, reliable way.

# Compute

Begin with input node w.

$$w = \text{chosen by user}, \quad \frac{dw}{dw}w = 1$$

Then move to the children nodes:

$$a \text{ and } \frac{da}{dw}, \quad b \text{ and } \frac{db}{dw}, \quad c \text{ and } \frac{dc}{dw}$$

$$\frac{da}{dw} = 1 - \tanh^2(w), \quad \frac{db}{dw} = -\sin(w), \quad \frac{dc}{dw} = \frac{1}{w}$$

# Compute

Node d = a * b (child of a and b). We need to use the product rule:

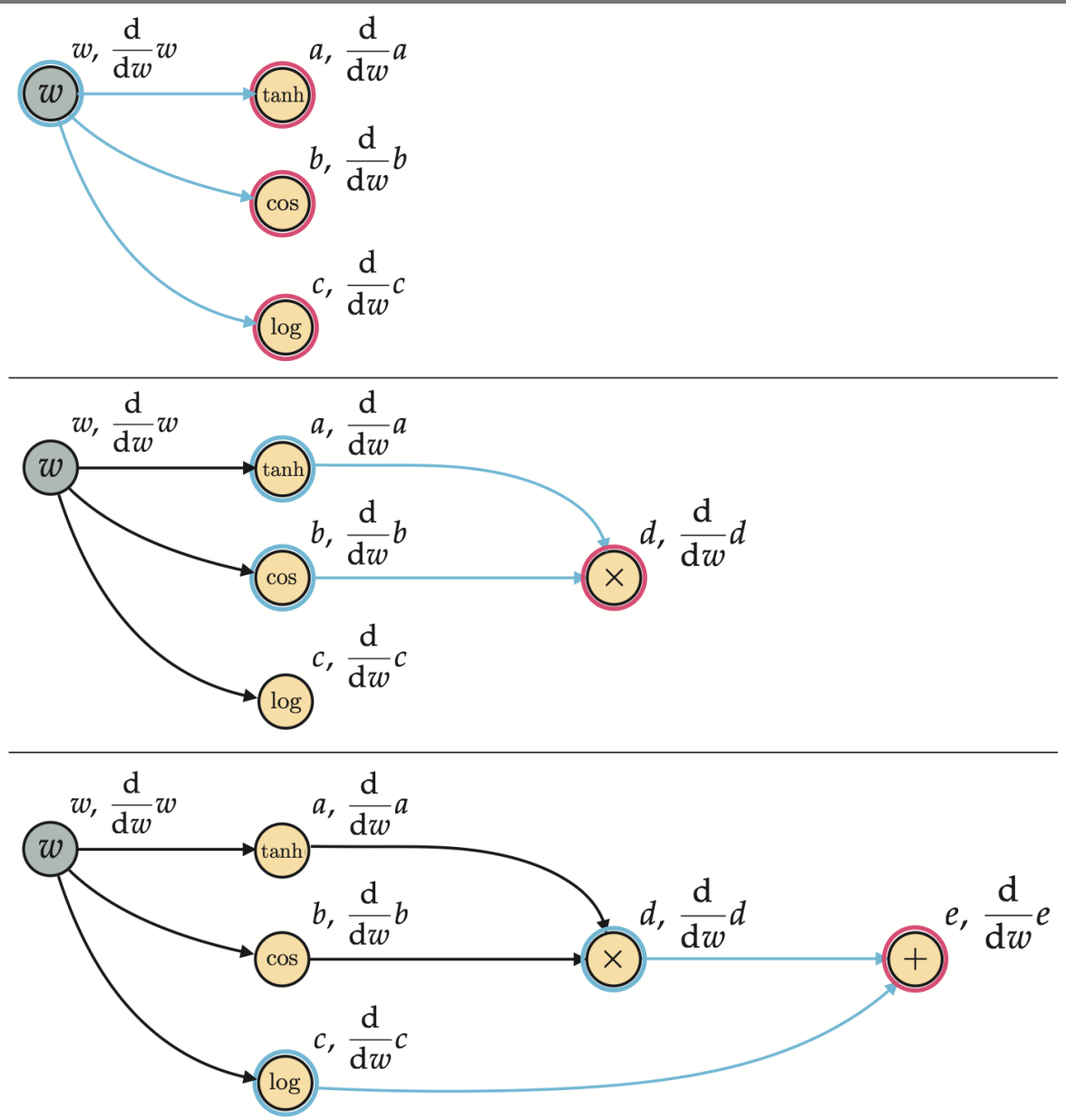
$$\frac{d}{dw}d(a,b) = \frac{\partial d(a,b)}{\partial a} \cdot \frac{da}{dw} + \frac{\partial d(a,b)}{\partial b} \cdot \frac{db}{dw} = b(w) \cdot \frac{da}{dw} + a(w) \cdot \frac{db}{dw}$$

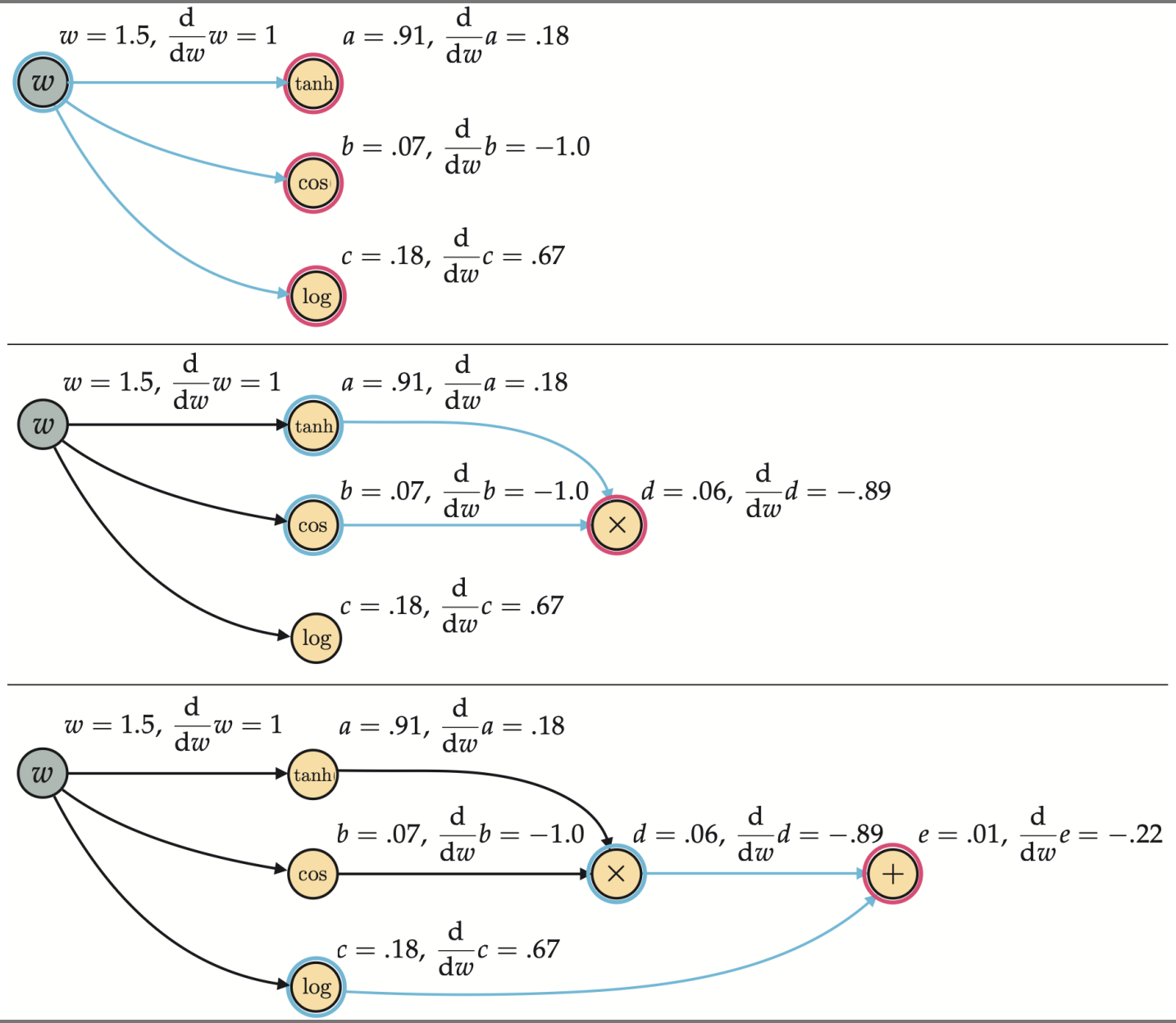
$$\frac{d}{dw}d(a,b) = \frac{da}{dw} \cdot b + \frac{db}{dw} \cdot a = (1 - \tanh^2(w))\cos(w) - \tanh(w)\sin(w)$$

For e = d + c we use the chain rule and the sum rule:

$$\frac{d}{dw}\, e\big(d(w), c(w)\big) = \frac{\partial e}{\partial d}\big(d(w), c(w)\big)\, \frac{dd(w)}{dw} \;+\; \frac{\partial e}{\partial c}\big(d(w), c(w)\big)\, \frac{dc(w)}{dw}$$

$$\frac{de}{dw} = \frac{dd}{dw} + \frac{dc}{dw} = ((1 - \tanh^2(w))\cos(w) - \tanh(w)\sin(w)) + \frac{1}{w}$$


w, d/dw w
a, d/dw a
b, d/dw b
c, d/dw c
d, d/dw d
e, d/dw e
w
tanh
cos
log
×
+


$w = 1.5,\ \frac{d}{dw}w = 1$
$a = .91,\ \frac{d}{dw}a = .18$
$b = .07,\ \frac{d}{dw}b = -1.0$
$c = .18,\ \frac{d}{dw}c = .67$
$d = .06,\ \frac{d}{dw}d = -.89$
$e = .01,\ \frac{d}{dw}e = -.22$
w
tanh
cos
log
×
+

# Issues with Forward AD

Each node computes **complete gradient w.r.t all inputs**

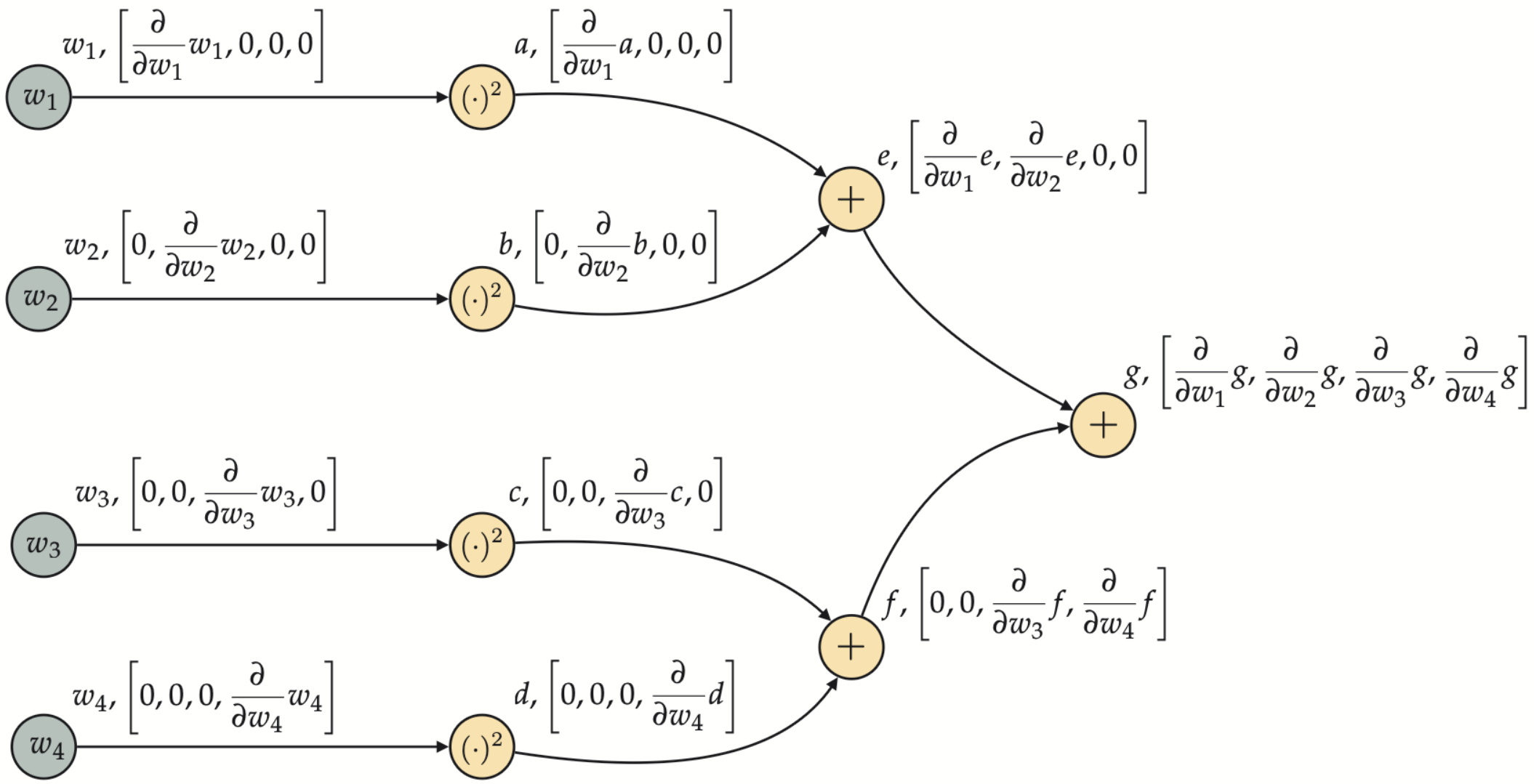
Example: Fully connected network → huge number of inputs

Leads to **computational waste** for large networks

**Reverse Mode Idea**

Traverse computation graph **forward**: compute only necessary derivatives

Then traverse **backward**: combine derivatives to construct full gradient


$w_1$
$w_1, \left[\frac{\partial}{\partial w_1} w_1, 0, 0, 0\right]$
$(\cdot)^2$
$a, \left[\frac{\partial}{\partial w_1} a, 0, 0, 0\right]$
$w_2$
$w_2, \left[0, \frac{\partial}{\partial w_2} w_2, 0, 0\right]$
$(\cdot)^2$
$b, \left[0, \frac{\partial}{\partial w_2} b, 0, 0\right]$
$+$
$e, \left[\frac{\partial}{\partial w_1} e, \frac{\partial}{\partial w_2} e, 0, 0\right]$
$+$
$g, \left[\frac{\partial}{\partial w_1} g, \frac{\partial}{\partial w_2} g, \frac{\partial}{\partial w_3} g, \frac{\partial}{\partial w_4} g\right]$
$w_3$
$w_3, \left[0, 0, \frac{\partial}{\partial w_3} w_3, 0\right]$
$(\cdot)^2$
$c, \left[0, 0, \frac{\partial}{\partial w_3} c, 0\right]$
$+$
$f, \left[0, 0, \frac{\partial}{\partial w_3} f, \frac{\partial}{\partial w_4} f\right]$
$w_4$
$w_4, \left[0, 0, 0, \frac{\partial}{\partial w_4} w_4\right]$
$(\cdot)^2$
$d, \left[0, 0, 0, \frac{\partial}{\partial w_4} d\right]$

# Reverse Mode

**Forward Pass**

- Start at input nodes, move left → right
- Compute only relevant partial derivatives

**Backward Pass**

Start from the **output node** and move recursively to the inputs. At each step:

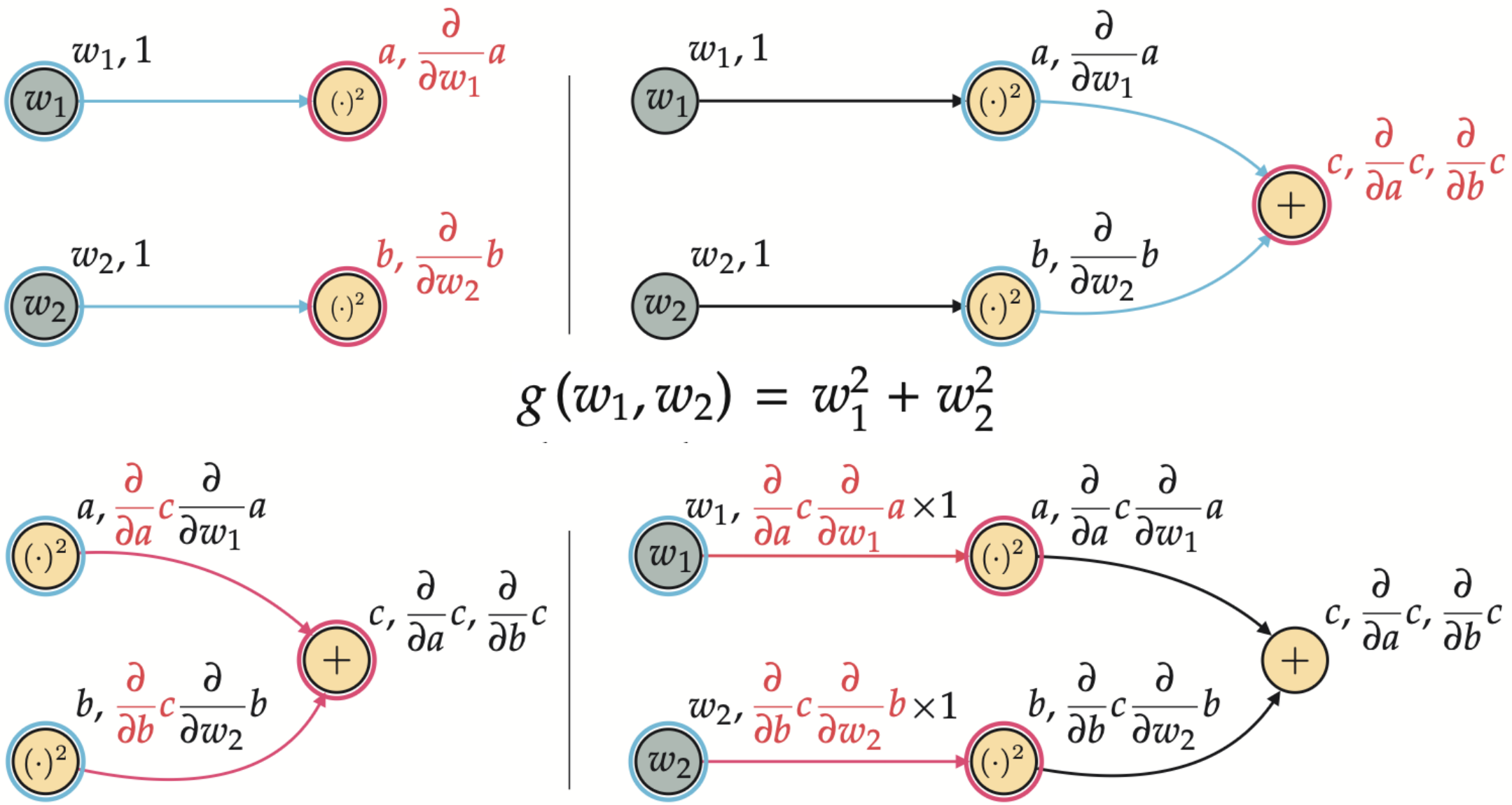
- Multiply the parent's partial derivative by the child's partial derivative (chain rule).
- If a node has multiple children, **sum the contributions** from each path.

**Pros**

- Much **less computation** for multi-input functions
- Standard in **machine learning applications**

**Cons**

- Must **store the computation graph**
- Slight memory overhead


$w_1, 1$
$w_1$
$(\cdot)^2$
$a, \frac{\partial}{\partial w_1} a$
$w_2, 1$
$w_2$
$(\cdot)^2$
$b, \frac{\partial}{\partial w_2} b$
$w_1, 1$
$w_1$
$(\cdot)^2$
$a, \frac{\partial}{\partial w_1} a$
$w_2, 1$
$w_2$
$(\cdot)^2$
$b, \frac{\partial}{\partial w_2} b$
$+$
$c, \frac{\partial}{\partial a} c, \frac{\partial}{\partial b} c$
$g(w_1, w_2) = w_1^2 + w_2^2$
$(\cdot)^2$
$a, \frac{\partial}{\partial a} c \frac{\partial}{\partial w_1} a$
$(\cdot)^2$
$b, \frac{\partial}{\partial b} c \frac{\partial}{\partial w_2} b$
$+$
$c, \frac{\partial}{\partial a} c, \frac{\partial}{\partial b} c$
$w_1$
$w_1, \frac{\partial}{\partial a} c \frac{\partial}{\partial w_1} a \times 1$
$(\cdot)^2$
$a, \frac{\partial}{\partial a} c \frac{\partial}{\partial w_1} a$
$w_2$
$w_2, \frac{\partial}{\partial b} c \frac{\partial}{\partial w_2} b \times 1$
$(\cdot)^2$
$b, \frac{\partial}{\partial b} c \frac{\partial}{\partial w_2} b$
$+$
$c, \frac{\partial}{\partial a} c, \frac{\partial}{\partial b} c$

# Backward Sweep: Two-Input Quadratic

1) Start at the output node **c** (end of forward pass).
2) Node **c** has two parents: **a** and **b**.
   - Update derivative at **a**: multiply the partial derivative by ∂c/∂a =1
   - Update derivative at **b**: multiply the partial derivative by ∂c/∂b =1

$$\frac{\partial g}{\partial w_1} = \frac{\partial c}{\partial a} \cdot \frac{\partial a}{\partial w_1} = (1)(2w_1)$$
$$\frac{\partial g}{\partial w_2} = \frac{\partial c}{\partial b} \cdot \frac{\partial b}{\partial w_2} = (1)(2w_2)$$

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}.$$

# Backpropagation in Practice

Reverse mode is the foundation of **autograd** in Python

Popular for deep learning frameworks (PyTorch, TensorFlow)

Efficient gradient computation for fully connected & deep networks

**Summary**

- Avoids wasted computation of irrelevant derivatives
- Forward pass → compute relevant derivatives
- Backward pass → construct gradient efficiently
- Essential for machine learning and neural network training

# Gradient Descent

The **negative gradient** of a function $g(w)$ at a point always gives a **valid descent direction**.

$$\mathbf{w}^{k} = \mathbf{w}^{k-1} - \alpha \nabla g\left(\mathbf{w}^{k-1}\right)$$

**Update Rule:**

- $\alpha$ :step size (learning rate)
- $\nabla g(w_{k-1})$ :gradient at current point

**Intuition:**

- Each step moves in the steepest descent direction.
- With an appropriate step size $\alpha$, this process leads toward a local minimum.

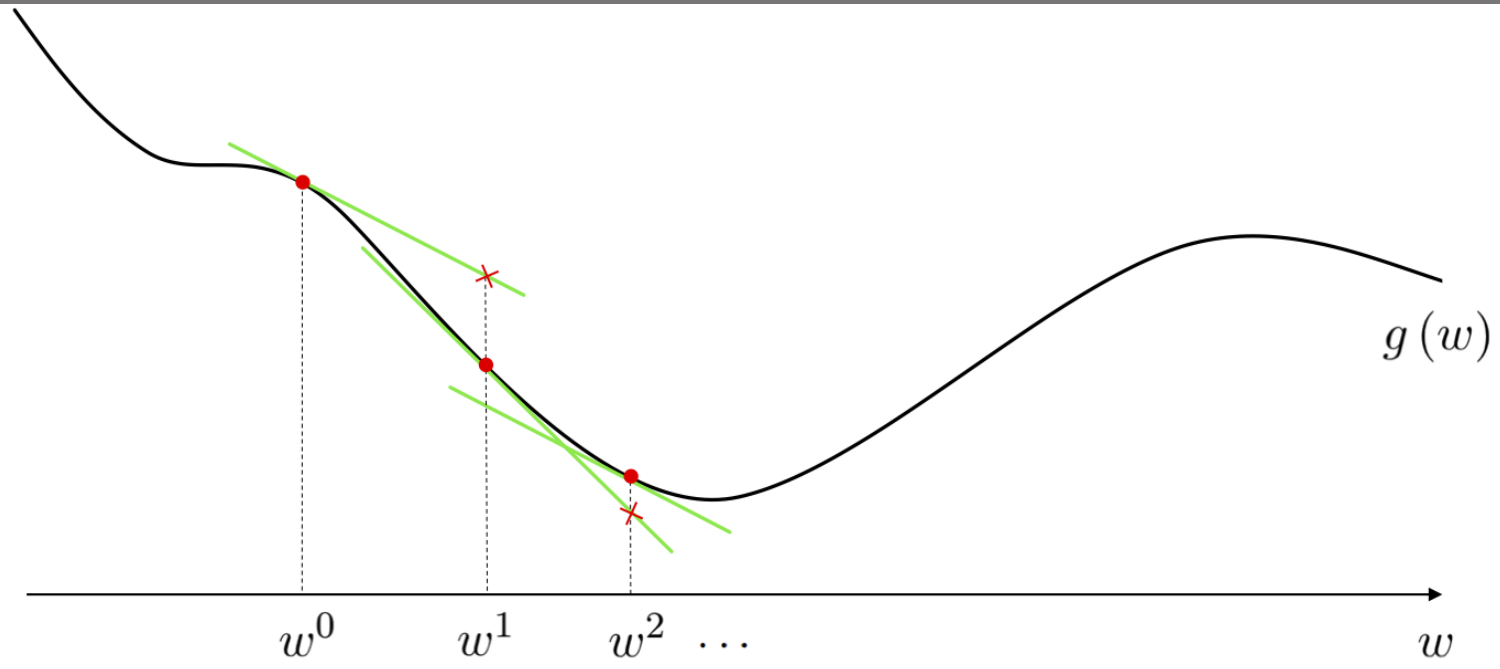
# Why it is popular

## Efficiency of computation

- Gradients are often *much easier to compute* than searching for random descent directions (zero-order methods).
- Automatic differentiation makes this process even simpler in high dimensions.

## Guaranteed descent direction

- The **negative gradient** always gives a **local descent direction**.
- Ensures progress toward a minimum, provided the step size is chosen properly.




Beginning at the point $w^0$, we make our first approximation is drawn below as a red dot, with the first order Taylor series approximation drawn in green.

Moving in the negative gradient descent direction provided by this approximation we arrive at a point $w^1 = w^0 - \alpha \frac{\partial}{\partial w} g\left(w^0\right)$

We then repeat this process at $w^1$ , moving in the negative gradient direction there, to $w^2 = w^1 - \alpha \frac{\partial}{\partial w} g\left(w^1\right)$ , and so forth.

# When does it stop?

If the step $\mathbf{w}^{k} = \mathbf{w}^{k-1} - \alpha \nabla g\left(\mathbf{w}^{k-1}\right)$ does not move from the prior point $\mathbf{w}^{k-1}$ significantly, it means that the direction we are traveling in is vanishing $-\nabla g\left(\mathbf{w}^{k}\right) \approx \mathbf{0}_{N\times 1}$

If step length $\alpha$ is chosen wisely, gradient descent halts **near stationary points**:

- **Global/Local minima**
- **Saddle points**

# Gradient Descent on Nonconvex Functions

**Challenge**

Gradient Descent may converge to **local minima** or **saddle points**, not necessarily the global minimum.

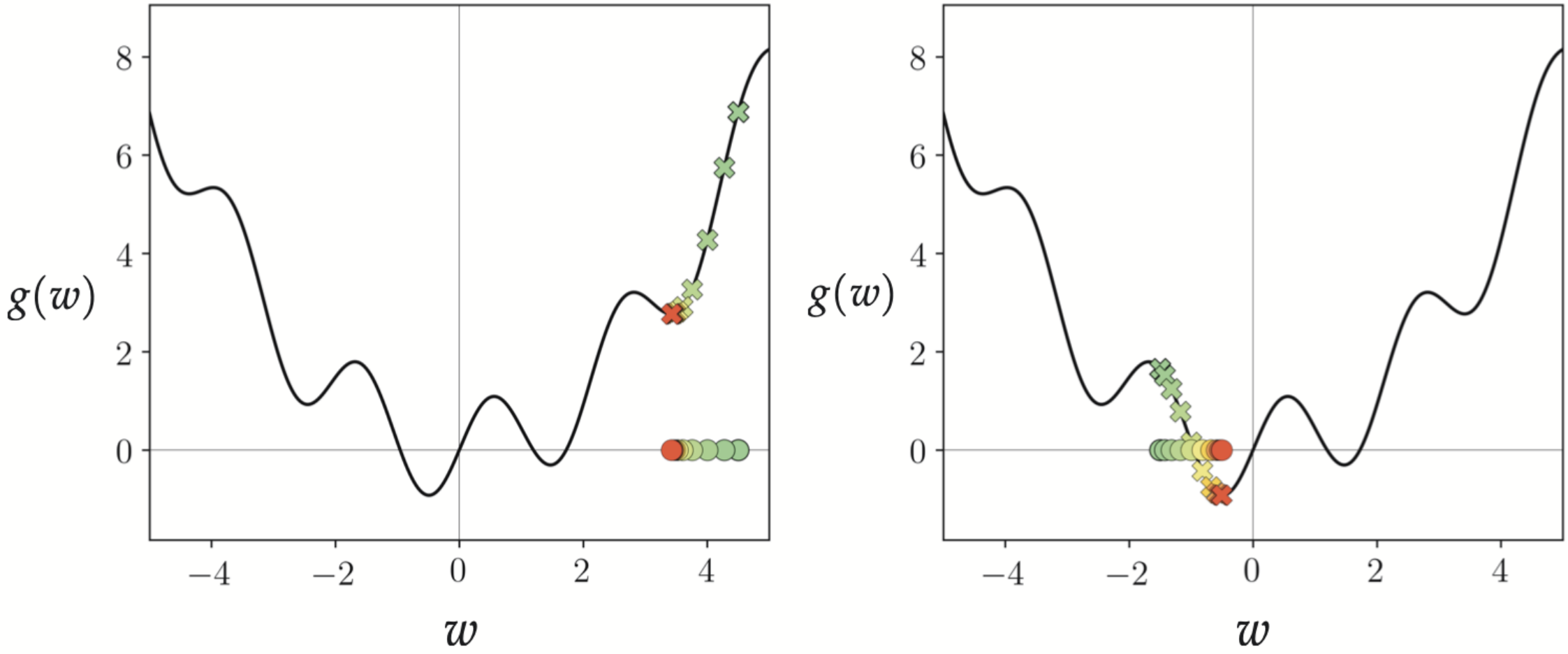
Outcome depends on:

- **Initialization** $w_0$
- **Step length** $\alpha$

**Example**

- Run with $w_0 = 4.5$ →local minimum
- Run with $w_0 = -1.5$ →global minimum

$$g(w) = \sin(3w) + 0.3w^2$$



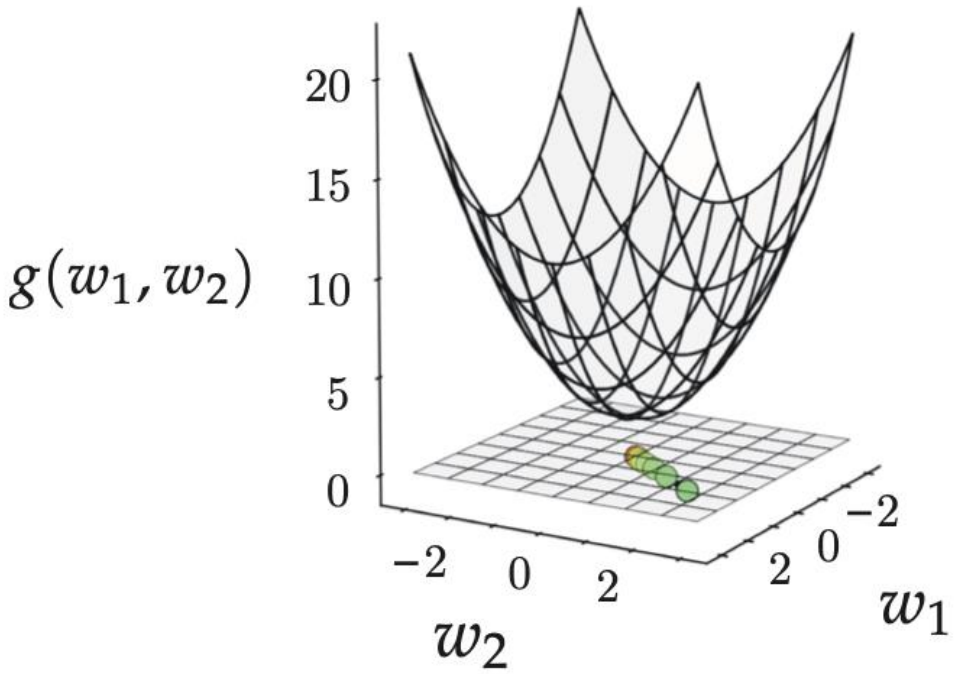
# Gradient Descent on Nonconvex Functions

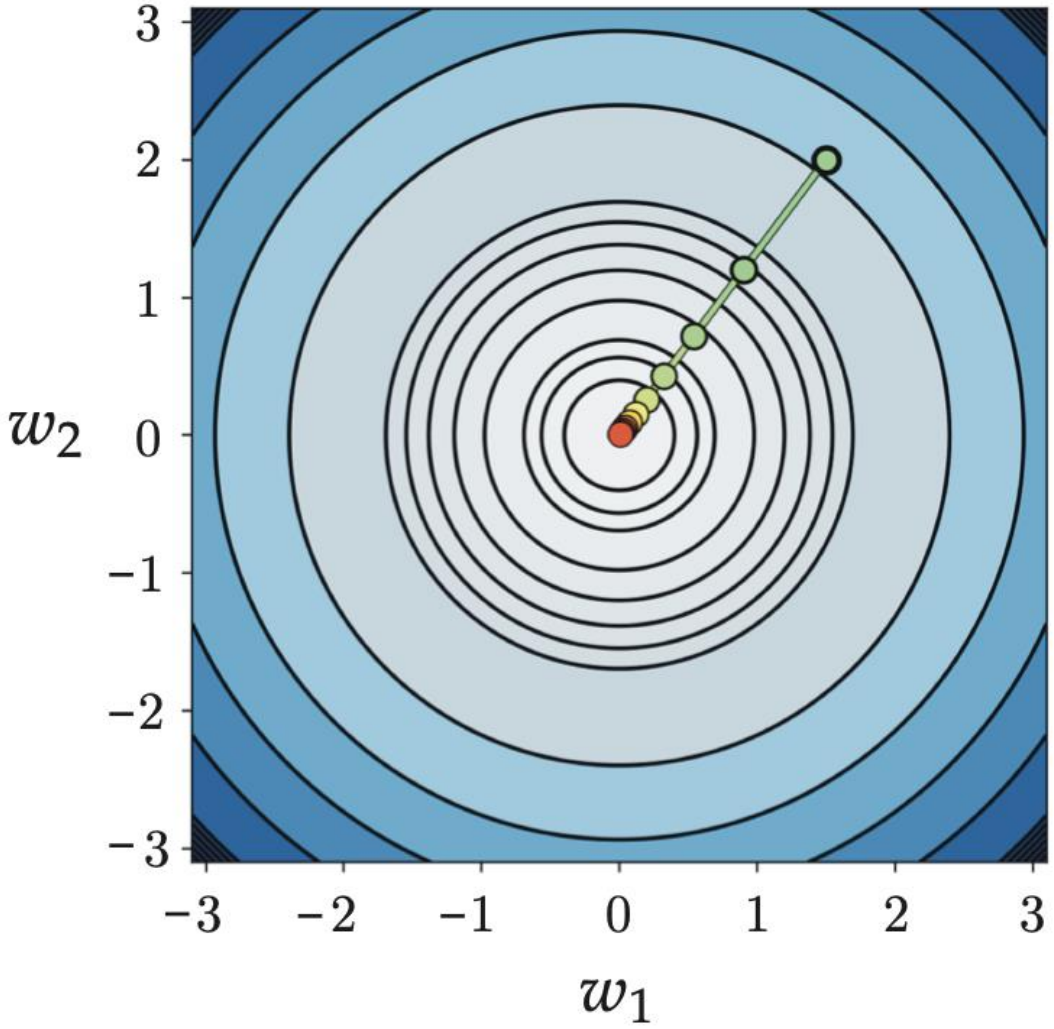
**Convex Case**

- For convex functions, all local minima are **global minima**.

**Example**

- $g(w_1, w_2) = w_1^2 + w_2^2 + 2$
- Gradient descent converges smoothly regardless of initialization.


$g(w_1, w_2)$
20
15
10
5
0
−2
0
2
$w_2$
−2
0
2
$w_1$


3
2
1
0
−1
−2
−3
$w_2$
−3
−2
−1
0
1
2
3
$w_1$

# Step size

**Key role of $\alpha$**

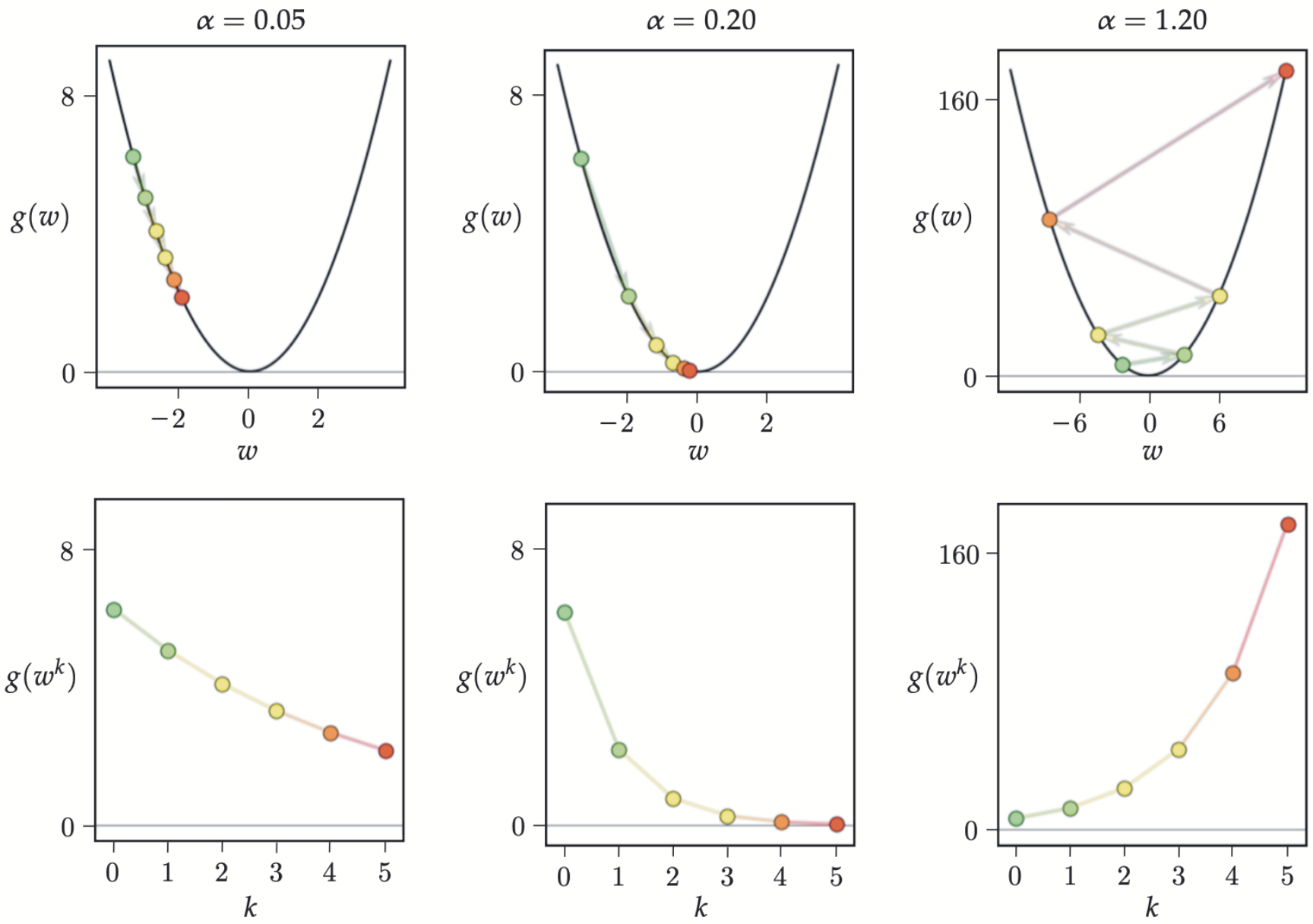
Too small → **very slow descent**.

Too large → **diverges / ascends**.

Needs to be chosen carefully.

**Basic strategies**

- **Fixed step length** ($\alpha = 10^{-m}$ ,with $m$ integer).
- **Diminishing step length** ($\alpha = 1/k$, at iteration $k$).


$\alpha = 0.05$
$\alpha = 0.20$
$\alpha = 1.20$
$g(w)$
$w$
$g(w^k)$
$k$
8
0
160
−2
2
−6
6
1
3
4
5

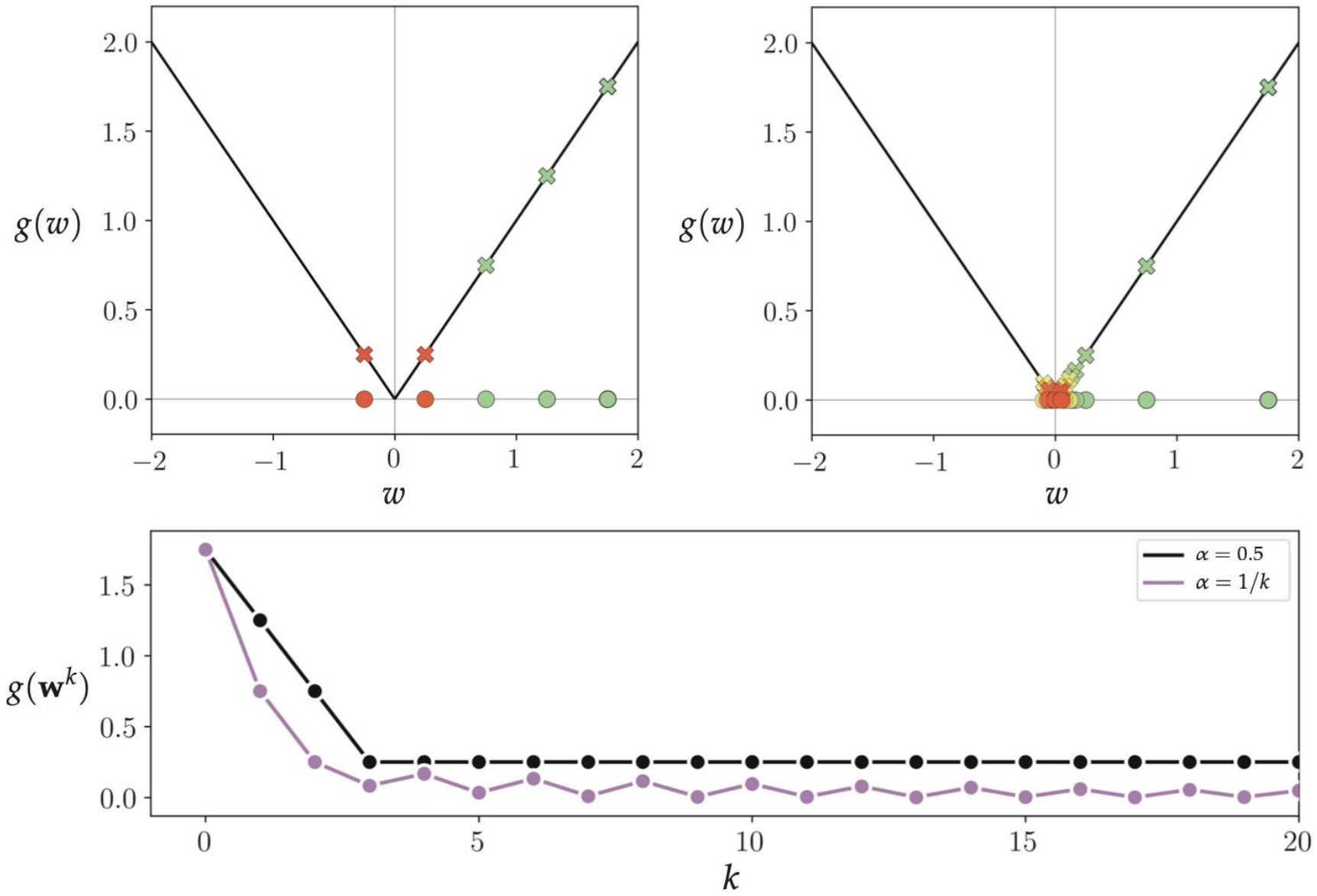
# Fixed vs Diminishing Step Length

**Example**

$$g(w) = | w |, w_0 = 1.75$$

**Fixed** $\alpha = 5 \times 10^{-1}$ **:**gets stuck, oscillates around 0.

**Diminishing** $\alpha = 1/k$**:** converges to global minimum.


$g(w)$
$w$
$g(w)$
$w$
$g(\mathbf{w}^k)$
$k$
$\alpha = 0.5$
$\alpha = 1/k$

# Cost Function History Plots

Plot of $g(w_k)$ vs. iteration $k$.

Shows how the cost function evolves during gradient descent.

**Why are they useful?**

- **Debugging tool**: detect divergence, oscillations, or stagnation.
- **Step length ($\alpha$) selection**: helps tune learning rate for stable and fast convergence (and many more hyperparameters).

In **high-dimensional functions** (where direct visualization isn't possible), cost history plots are often the *only practical tool* for monitoring optimization progress.

# Oscillation in Cost Function History Plots

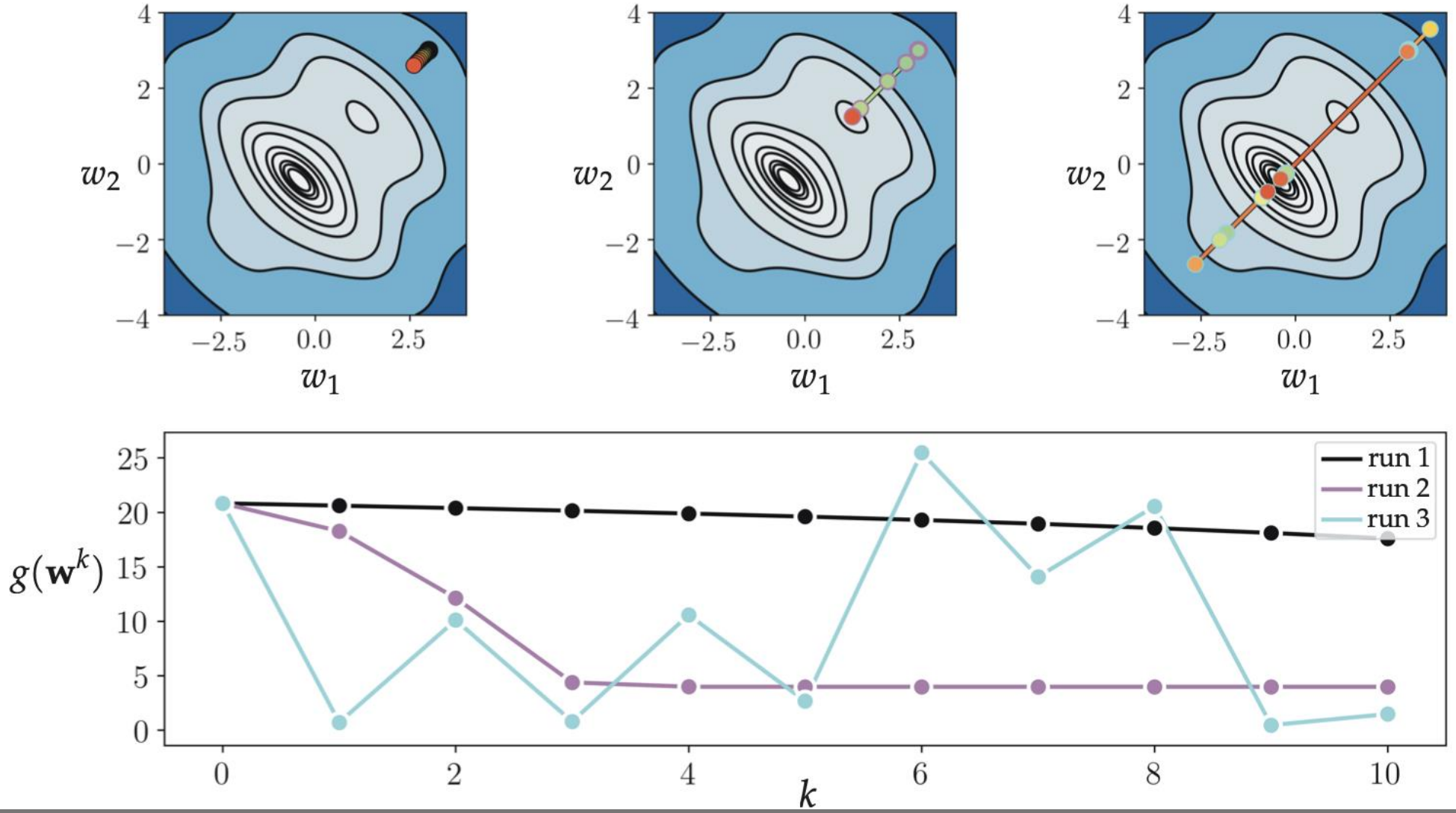
A cost history plot need **not** be strictly decreasing at every step.

What matters: reaching the **lowest possible function value**.

Some **oscillation is acceptable** (and often desirable) if it helps escape shallow minima and reach a lower cost.

The best $\alpha$ may not yield a smooth, monotone curve.

$$g(\mathbf{w}) = w_1^2 + w_2^2 + 2\sin(1.5(w_1 + w_2))^2 + 2$$



# Implications for Step Length ($\alpha$):

- **Too small** $\alpha = 10^{-2}$: very slow progress.
- **Moderate** $\alpha = 10^{-1}$: converges, but may settle in a **local minimum**.
- **Large** $\alpha = 10^{0}$: oscillatory, “hopping around,” but may reach a **better (global) minimum**.

# When do we stop?

Halts near **stationary points** (local minima or saddle points).

$$w_k = w_{k-1} - \alpha \nabla g(w_{k-1}) \;\Rightarrow\; w_k \approx w_{k-1} \;\Longrightarrow\; \nabla g(w_k) \approx 0$$

**1) Gradient-based criterion:**

$$\| \nabla g(w_{k-1}) \|_2 \;\; \textit{sufficiently small} \;\Rightarrow \textit{near stationary point}$$

**2) Step-based criterion:**

$$\frac{1}{N} \| w_k - w_{k-1} \|_2 \leq \epsilon \;\Rightarrow \textit{steps no longer make progress}$$

**3) Function-value criterion:**

$$\frac{1}{N} | g(w_k) - g(w_{k-1}) | \leq \epsilon \;\Rightarrow \textit{cost changes negligibly}$$

**4) Practical Criterion:**

Stop after a **maximum number of iterations.**

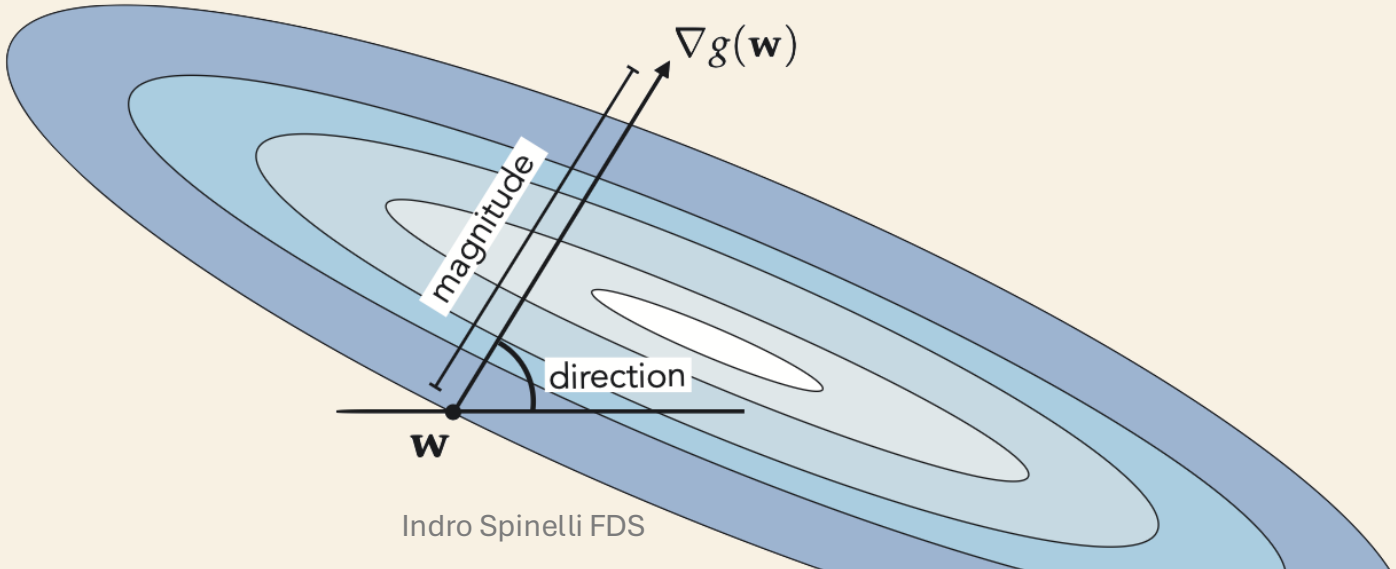
# Weaknesses of GD

Every gradient has a **magnitude** and a **direction**.

Problems in gradient descent stem from either or both:

- **Direction** – can oscillate (zig-zagging).
- **Magnitude** – can vanish near stationary points (slow-crawling).

Gradient is always **perpendicular to function contours**.



# Zig-Zagging Behavior

In **narrow valleys**, contour lines are nearly parallel.

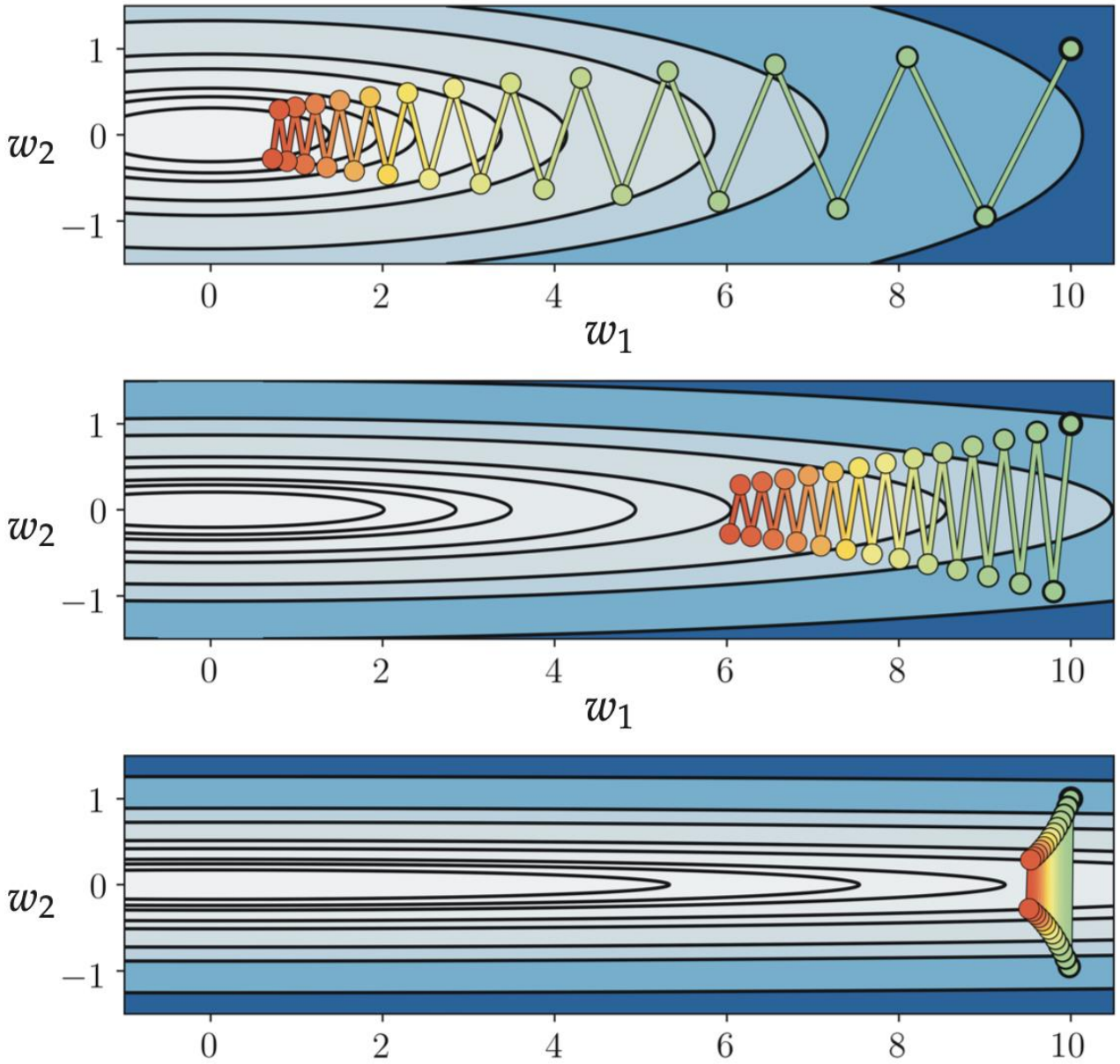
Gradient points perpendicular → successive steps alternate direction.

**Effect**

- Steps **zig-zag**, slowing convergence.
- Reducing $\alpha$ can help but does not fully solve the problem.

**Example**

- Quadratic $g(w) = w^T C w$ with elongation along one axis.
- Same initialization, larger elongation → more zig-zagging.


$w_2$
1
0
−1
0
2
4
6
8
10
$w_1$
$w_2$
1
0
−1
0
2
4
6
8
10
$w_1$
$w_2$
1
0
−1
0
2
4
6
8
10

# Slow-Crawling Near Stationary Points

**Near minima or saddle**

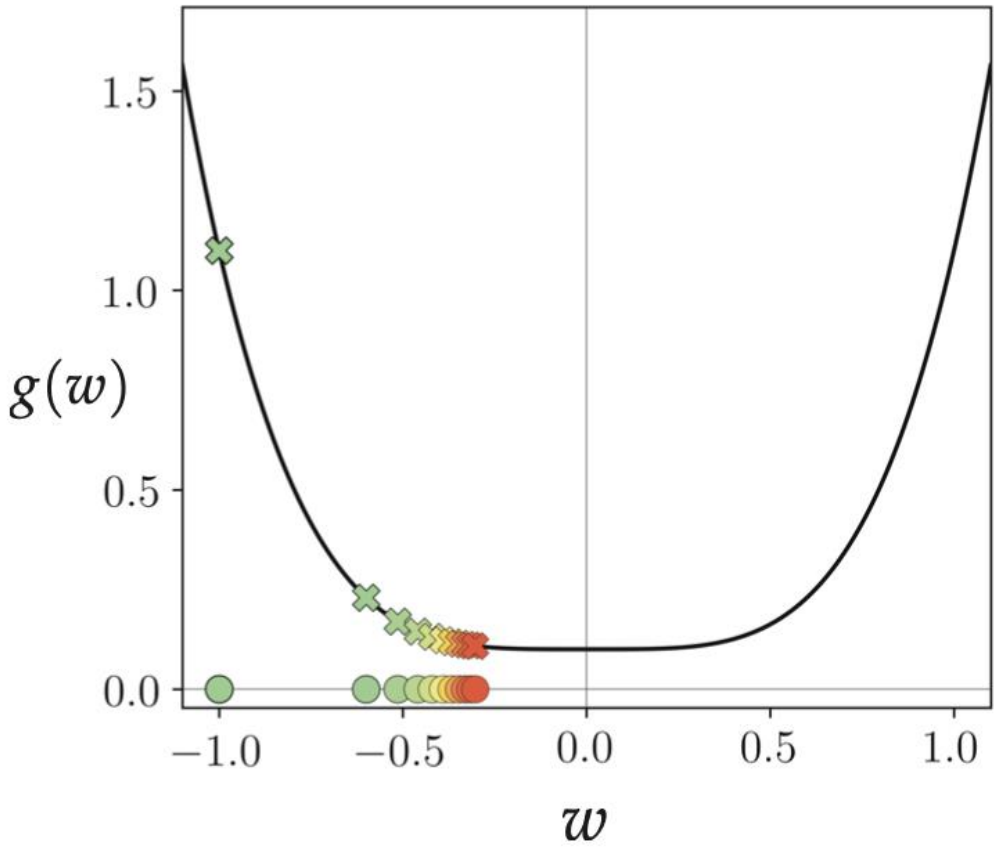
Small gradient: $\| \nabla g(w) \|_2 \approx 0$ → very small step $\| w_k - w_{k-1} \|_2$

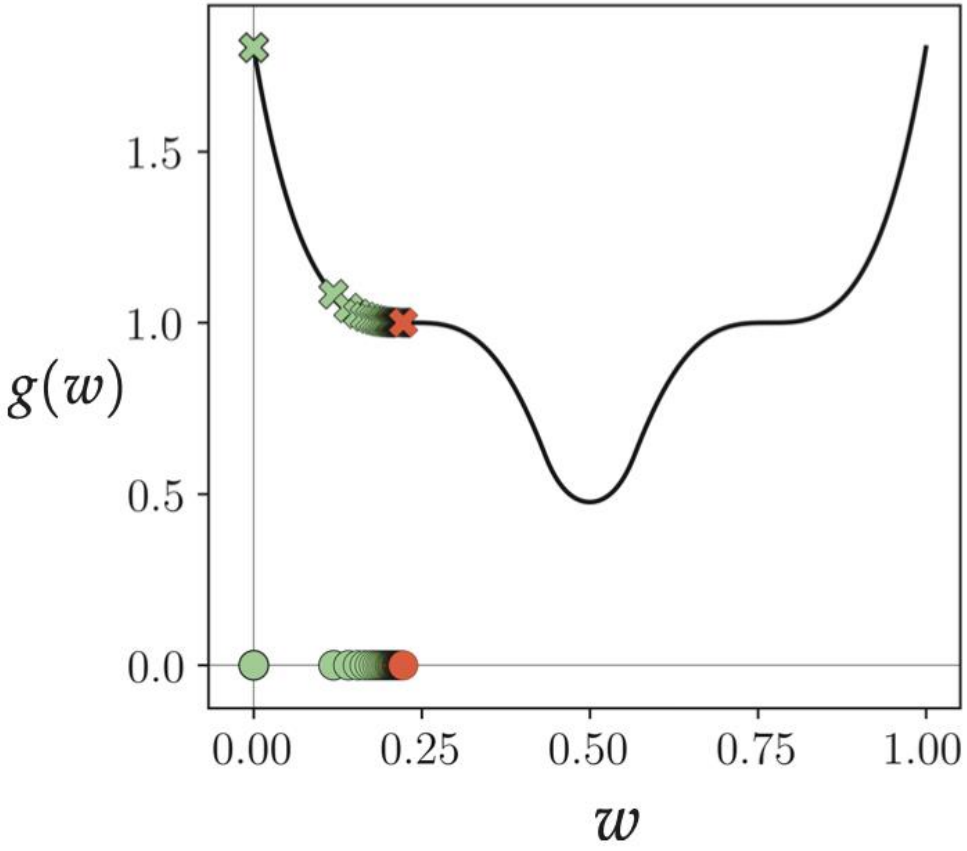
**Effect**

- Gradient descent slows near minima or saddle points.
- Can halt near saddle points in nonconvex functions, missing global minima.

**Example**

- $g(w) = w^4 + 0.1$ →steps large initially, crawl near minimum.
- Nonconvex function → steps halt at saddle point.


1.5
1.0
0.5
0.0
$g(w)$
−1.0
−0.5
0.0
0.5
1.0
$w$


1.5
1.0
0.5
0.0
$g(w)$
0.00
0.25
0.50
0.75
1.00
$w$

# Gradient Descent for PCA

**Variance Maximization**

Objective: maximize total variance along all components:

$$\max_{C} tr(C^T XX^T C) \text{ s.t. } C^T C = I_K$$

Gradient descent update (negative gradient of $-\mathrm{tr}(C^T XX^T C)$

$$C_k = C_{k-1} + \alpha XX^T C_{k-1}$$

**Constraint**

orthonormalize columns of $C_k$ after each step: $C_k^T C_k = I_K$

# Gradient Descent for PCA

**Reconstruction Error Approach**

Objective: minimize reconstruction error:

$$L(C) = \| CC^T X - X \|_F^2$$

Gradient descent update:

$$\boxed{\nabla_C L(C) = 2(CC^T X - X)X^T C}$$

$$C_k = C_{k-1} - \alpha \nabla_C L(C_{k-1})$$

**Constraint:** orthonormalize columns of $C_k$ after each step.

# Chapters 2,3 and Appendix B of Machine Learning Refined Book